

OctoVIZN x MPGI Attendance Dashboard: Backend Integration Guide

Audience: backend and AI-model team **Frontend files:** `dashboard.html`, `dashboard.js`, `dashboard.css` (plain HTML/JS, no framework, no build step)

1. What the dashboard does today

The operator picks a **video source** (an MP4/MOV file or an RTSP URL), picks a **class**, and clicks **Mark attendance**. The roster then shows each student as present or absent. The operator can download the results as JSON or CSV.

It is **currently frontend-only**. `runAttendance()` in `dashboard.js` fakes the result with `Math.random()`. Nothing is uploaded and no API is called. Your job is to replace that fake step with the real flow below.

2. What the frontend sends

Item	Detail
Source: file	One MP4 or MOV, max 2 GB (validated in the browser)
Source: RTSP	A URL matching <code>rtsp://</code> or <code>rtsp://</code> , e.g. <code>rtsp://user:pass@host:554/stream</code>
Class	A string such as "Class A"
Operator	Name from <code>localStorage (trace_user_name)</code> , for audit only

Only one source is active at a time.

3. What the frontend needs back

For each student in the selected class:

```
{ "name": "Aditi Sharma", "status": "present", "confidence": 0.93 }
```

- `status` is "present" or "absent". You decide the confidence threshold.
- `confidence` is a number from 0 to 1. Always return it. The UI shows it only if the operator enables "Include confidence values".
- Return **every** student on the class roster, including those not detected (mark them `absent`).

The exported JSON has this shape, so keep the same shape in your response:

```
{
  "class": "Class A",
  "date": "2026-10-04T09:30:00.000Z",
  "source": { "type": "mp4", "name": "lecture.mp4", "size": 183746291 },
  "students": [ { "name": "...", "status": "present", "confidence": 0.93 } ]
}
```

4. Suggested API

Base URL is configurable (we will add an `API_BASE` constant in `dashboard.js`). All endpoints over HTTPS.

1. GET `/api/classes` returns `{ "Class A": [{ "id": "s101", "name": "Aditi Sharma" }, ...], ... }`. This replaces the hard-coded `CLASSES` object. The 8 current classes and their students are in the top of `dashboard.js`.
2. POST `/api/attendance/jobs` starts a run.
 - File source: `multipart/form-data` with fields `class`, `operator`, `video`.
 - RTSP source: JSON `{ "class", "operator", "rtsp_url" }`.
 - Returns `{ "job_id": "..." }`.
 - Because files can reach 2 GB, a resumable or chunked upload, or a presigned URL to object storage, is recommended over a single request. Tell us which and we will adapt.
3. GET `/api/attendance/jobs/{job_id}` returns `{ "status": "queued|running|done|failed", "progress": 0.0-1.0, "results": [ ...per-student objects as in section 3... ], "error": null }`. The frontend will poll every 1-2 s. Server-sent events are fine if you prefer. If you stream per-student results as they are decided, the roster animates live as it does now.
4. POST `/api/feedback` is optional. See section 6.

5. AI pipeline expectations

- Reference photos live **server-side**, one folder per class. Student names or IDs must match the roster returned by `/api/classes`. We recommend matching by stable `id`, not by name string.
- The backend extracts frames from the video, detects and recognises faces, and matches them against that class's reference photos only.
- **Open decision, RTSP**: a live stream has no end. Please define the behaviour, for example "sample for N minutes then finish". If you want an operator-controlled **Stop** button, tell us and we will add it.

6. Other items to wire up

- **Feedback form**: `FORMSPREE_ENDPOINT` in `dashboard.js` is a placeholder (`.../f/yourFormId`). Either supply a real Formspree ID or give us your own endpoint. The form posts `name` and `message` as multipart form data.
- **Login**: the name and password on `index.html` are stored in the browser's `localStorage` only. This is a convenience lock, not real security. If real authentication is required, we will need a login endpoint and a token on every API call.
- **Static pages**: `howtouse.html`, `privacy.html`, `terms.html` and `about.html` are linked from the settings menu and must exist alongside the dashboard.

7. Security and privacy

- **RTSP credentials**: the URL may contain a username and password. The dashboard masks the password on screen, but the full URL is sent to you. Accept it over TLS only, never write it to logs, and store it encrypted if you persist it.
- **Face data and video are sensitive**. Define retention (we suggest deleting uploaded videos after the job completes) and restrict access to reference photos.

- **CORS and HTTPS:** If the API is on a different origin from the dashboard, enable CORS for the dashboard's origin. If the dashboard is served over HTTPS, the API must be too.
- **Upload limits:** raise any proxy or server body-size limits to at least 2 GB, or use the chunked approach in section 4.

8. Handoff checklist

- ☐ API base URL and chosen upload method
- ☐ Confirmed response shape (section 3)
- ☐ RTSP run-length behaviour decided
- ☐ Reference photos loaded per class and matched to roster IDs
- ☐ Formspree ID or feedback endpoint supplied
- ☐ CORS and HTTPS configured

Once the endpoints exist, the frontend change is small: swap `runAttendance()` for upload, poll and render, and swap `CLASSES` for `GET /api/classes`.

